

Can a local LLM replace cloud Claude Code?

Three weeks, three machines: from a Raspberry Pi to a \$4,676 Mac Studio.

TL;DR bounded coding **ties the frontier, free**. Open-ended repos: **still cloud**.

Agenda (~29 min)

Act	Topic	~min
1	The route: Raspberry Pi, Air, Studio	7
2	Reality check: you can't buy parity. The \$4,676 call	5
3	The measured verdict: one-shot, agent loop, the build test	12
4	Economics, recommendation, caveats	5

ACT ONE

The route: three machines

The question

- Can I run a 'SOTA' local LLM at home, usable as my Claude Code backend, and stop paying for cloud?
- Two things to find out:
 - Is it possible, and at what hardware/cost?
 - Is it good enough, measured, not vibes?

The route: three machines, three verdicts

Stop	Hardware	Verdict
1	Raspberry Pi 5 + AI HAT+ 2 (Hailo-10H)	Dead end: no API for Claude Code (~2B / small context anyway)
2	MacBook Air M2 16GB ('Maral')	Did the grunt work (~5 hrs). qwen3:14b ~10 tok/s, kinda slow
3	The tuning wall	think:false = ~3x; quant sweep; prompt slim
4	Reality check	Frontier parity NOT purchasable locally at any price
5	Mac Studio M3 Ultra 96GB (\$4,676)	Ties Opus on coding @ 68 tok/s, \$0/mo

Dead end #1: the AI HAT+ 2 couldn't talk to Claude Code

- The AI HAT+ 2 (Hailo-10H, 40-TOPS (tera-operations per second) NPU (neural processing unit), 8GB on-board) is a real generative-AI accelerator, it runs small LLMs on-chip, not just vision.
- **The wall:** it speaks Hailo's own runtime, not an Anthropic- or Ollama-style endpoint. Claude Code had nothing to connect to, so the tool-use loop never wired up.
- Even past that: Hailo's supported models are ~1-2B class with a small context, far short of the 64k+ a coding agent needs.
- **Lesson:** the blocker was the API surface, not the silicon. A brilliant accelerator with no drop-in endpoint is still a dead end for an agent.

Showing the work: it'd be too small anyway

- Integration was the real wall (last slide). But say it had an endpoint, the ceiling is still low. Napkin math:
- The model lives in the Hailo-10H's 8GB on-board memory. Max model size is roughly $\text{on-board RAM} / \text{bytes-per-parameter}$.
 - At Q4 (4-bit quantization) that's ~0.5 bytes/param: 8GB caps you near ~10B in theory, and Hailo's supported set is smaller in practice, ~1-2B class.
 - Decode speed follows the same rule: $\text{tok/s} \sim \text{memory bandwidth} / \text{active model bytes}$ (the same formula that makes the Mac Studio fast later).
- **Bottom line:** a ~1-2B model with a small context can't be a 64k-context coding agent. Right silicon, wrong job.
- Honest caveat: I didn't push further, the Mac path was clearly better, so this is back-of-envelope, not an exhaustive benchmark.

Maral: a spare 16GB Air, doing the most

- qwen3:8b / :14b via Ollama's Anthropic endpoint, wired into Claude Code with one env block
- Plot twists:
 - Tool-use was already at parity with cloud. The worry? Misplaced.
 - Real pain wasn't the model, it was memory bandwidth and the WiFi driver crashing under load (rude)
- 16GB is the floor, not a platform. Ran on vibes and about 5 hours of uptime.

The single best tuning knob: kill the thinking tax

- Qwen3 emits a `<think>` trace before every answer.
- A coding answer needing 80 tokens cost 600, 8x the work.
- `CLAUDE_CODE_DISABLE_THINKING=1` (or `{"think": false}`)
- **~3x speedup from one env var.** Everything else I tuned for a week? Secondary.

ACT TWO

Reality check & the \$4,676 call

Reality check: you can't buy frontier parity

Tier	Best model	SWE-bench	Gap to Opus 4.8
96GB Mac	qwen3-coder:30b	~77%	-12
192GB	qwen3-235b Q4	~86%	-3
512GB (\$11.5K)	DeepSeek-V3 Q4	~88%	-4 (still short!)
Cloud	Opus 4.8	88.6%	0

Open weights trail the closed frontier by 6-12 months. **The only thing that gives you Opus quality is Opus.**

So the decision is about how close, not parity

- Don't chase a number that isn't for sale
- **Buy hardware for the 80%; keep an explicit `claude-cloud` for the 20%**
- Denominate the choice in your actual task mix, not dollars or principle
- Sweet spot for one dev: **~\$4-5K, one Mac Studio, 96GB**

The buy: \$4,676 for a used M3 Ultra 96GB

- Apple-direct was 4 months backordered (M3 Ultra was EOL'd)
- Every reseller went dry within days, only the grey market left
- Verified a sealed eBay unit (99.3% seller); \$4,299 + tax. Wiring \$4,676 to a stranger is its own personality test.
- **Lesson:** a just-discontinued machine vanishes from every channel at once.



the box running the LLM

ACT THREE

The measured verdict

The verdict: local ties cloud on coding

Model	Score	Speed
qwen3-coder:30b (local)	24 / 24	68 tok/s
Opus 4.8 (cloud)	24 / 24	—
qwen3:32b dense	16 / 18	20 tok/s (skip)

Mini-bench: 24 algorithmic problems, easy to LeetCode-hard, deterministic pytest scoring. Local understood the assignment: tied Opus on every one.

What the tie means

- **Local owns:** self-contained coding, functions, scripts, algorithms, single and moderate multi-file
- **Cloud still wins:** open-ended, multi-file, sprawling-context repo work (real SWE-bench, the software-engineering benchmark)
- The gap is real but lives on a different axis than most benchmarks test.
- A bench your best model can't lose on has stopped measuring.

Why the Studio is fast: bandwidth, not parameters

Box	Memory bandwidth	coder-30b speed
MacBook Air M2 16GB	~100 GB/s	16 tok/s
Mac Studio M3 Ultra 96GB	~800 GB/s	68 tok/s

Same model, 4x faster, entirely the memory bus. **MoE (mixture of experts) beats dense:** dense 32B is mid; the 3B-active MoE ate, 3x faster and higher-scoring.

Bonus: one box = a full local AI server

With `OLLAMA_MAX_LOADED_MODELS=3`, all resident at once (~38GB, 50GB free):

- coder-30b, the coding agent
- qwen2.5-VL, vision / OCR
- nomic-embed, RAG (retrieval-augmented generation) embeddings

Plus qwen3-next:80b (80B, 64 tok/s, bigger brain, same speed).

But the agent loop is where local gets cooked

	pass	avg wall-clock	turns (range)
Cloud (Opus)	5 / 5	31 s	5-9 (stable)
Local (coder-30b)	4 / 5	248 s	4-41 (wild)

5 multi-file bug-fixes through the real Claude Code agent loop. **Local 8x slower, a one-line `>` to `>=` fix took it 41 turns / 584 s.**

Watch it: the agent loop side by side

Same one-line bug · the real Claude Code agent loop task: ttl_off_by_one (cache expiry > → >=) 8x real time

● LOCAL · qwen3-coder:30b turn 3 · Mac Studio

28.4s

- Read(cache.py)
- Read(store.py)
- Bash(pytest -q 2 failed)

● CLOUD · Opus 4.8 turn 5 · Anthropic

17.3s

- Read(cache.py)
- Read(test_task.py)
- Edit(cache.py expiry > now → >= now)
- Bash(pytest -q test_task.py)
- ✓ 4 passed in 0.04s

PASS ✓ 5 turns · 17.3s

cloud done: 17.3s / 5 turns local: 110.4s / 9 turns – 6.4x slower for the SAME fix replay

DATA SOCIETY:

It's not caching, it's convergence

- Ollama DOES prefix-cache (the big local token counts are CC's accounting, not the box re-computing)
- The real cost is turn-count instability: the 30B fumbles, sometimes spiraling to 41 turns, sometimes giving up at 4
- Cloud converges in 5 turns every time. Local swings 4-41.
- The one-shot '68 tok/s, ties Opus' number was single-turn, it hid all of this.

The reconciliation

Workload	Local verdict
One-shot bounded coding	Ties Opus , fast, free
Multi-turn agent loops	Works (4/5) but ~8x slower, unstable
Open-ended multi-file repos	Loses on capability too

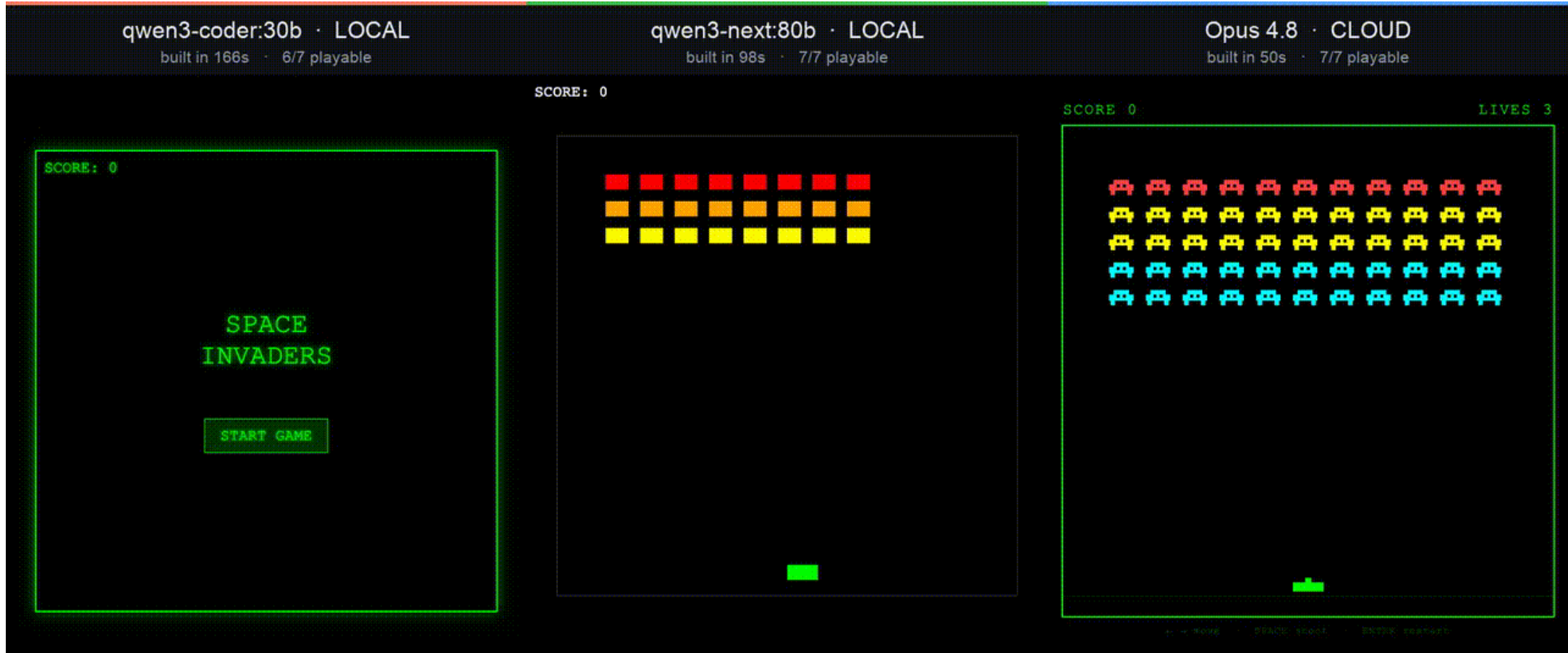
Local is capable AND the daily agent experience is far less reliable than raw speed implied. Both true.
Measure the agent loop, not just tok/s.

But on open-ended building, the gap collapses

backend	playable	build time	turns	cost
Opus 4.8 (cloud)	7 / 7	50 s	2	\$0.40
qwen3-next:80b (local)	7 / 7	98 s	2	\$0
qwen3-coder:30b (local)	6 / 7	166 s	2	\$0

Task: 'build playable Space Invaders as one index.html', scored by a Playwright 7-check rubric. **No failing test to spiral on, so local 80B ties cloud, within 2x on speed.**

Same task, three models, all playable



ACT FOUR

Economics & the call

The economics

- Cloud: ~\$200/mo, about \$2,400/yr, indefinitely
- Local: \$4,676 once, then **\$0/mo** after
- **Breakeven ~2 years**, then it's basically free real estate, for the work local handles well
- Privacy bonus: code never leaves the LAN
- The catch: it's a supplement, not a full replacement. Hard repo work still routes to cloud.

Recommendation

Hybrid, not either/or:

- `claude` routes to the local Studio (the 80%: daily coding, free, private, fast)
- `claude-cloud` routes to Opus 4.8 (the 20%: hard cross-file repo work)

One toggle, per task. Denominated in your real workload.

Single-dev sweet spot: Mac Studio, **64-96GB**, MoE coder model, ~\$4-5K.

Caveats that matter

- 'Local SOTA at home' is real in 2026, narrowly, on bounded coding
- On open-ended engineering it is not, and no honest setup pretends otherwise
- 16GB is too small; 96GB+ is overkill for one person
- Benchmarks saturate, measure your task mix, not a leaderboard

What's next (this is being measured)

- Cut my agents fully over to local
- Measure local vs Opus 4.8 on real tasks: token usage, latency, TTFT (time to first token), success
- Add verification-loop scaffolding: guardrails took an 8B from 53% to 99% on agentic workflows
- The instability may be a software problem, not a model-size one. The fix might be code, not a \$10k box.

The win is real. So is the caveat.

Repo + full write-up: github.com/jconnolly/local-llm-pi5

Benchmarks: minibench · repobench · appbench

PS: the field moved while I built this deck

- Mid-build, two things landed the same week: Vicki Boykis's *Running local models is good now* (Jun 15) and a 1,589-point Hacker News thread.
- They corroborate the core finding: context is the wall, ~30B MoE is the sweet spot, agentic local is real but model-dependent.
- But my models are already last-gen, the community moved to **Qwen 3.6** (27b / 35b-a3b) and **Gemma 4**; my coder-30b is a step behind.
- A likely fix for the instability: q4 quant weakens tool-calling, **q6** is the agentic sweet spot. My 80B ran at q4, that may be the bug, not the model.
- **Next:** re-bench Qwen 3.6 + Gemma 4 + gpt-oss, re-run the agent loop at q6. The verdict holds; the models won't, this moves weekly.
- Sources: [blog](#) + [HN](#)